Attendance

Stallings, Computer Organization & Architecture (9th Ed.), Chapter 9
Hamacher, Vranesic, & Zaky, Computer Organization (5th Ed.), Appendix E

Computer Organisation

Topic 2

Number Systems

Stallings, Computer Organization & Architecture (9th Ed.), Chapter 9
Hamacher, Vranesic, & Zaky, Computer Organization (5th Ed.), Appendix E

1234

- What do you see?

Outline

Decimal systems

Binary systems

Converting between binary and decimal

Converting between hexadecimal and binary

Character Codes

Introduction

- One of the characteristics of **digital** systems:
  - To represent and manipulate **discrete** elements of information (10 decimal **digits**, 26 alphabet **letters**, 64 **squares** of chessboard)

Number systems

- Decimal Number System (Base-10) – commonly used
- Binary Number System (Base-2) – two values ONLY
- Octal Number System (Base-8) – commonly used in computer programming context
- Hexadecimal Number System (Base-16) – commonly used in CS and programming
- Other Number Systems
  - Base-12 (Duodecimal)
  - Base-60 (Sexagesimal)
  - Base-64 (encoding data for transmission)

Types of number system

- Binary
- Octal
- Decimal
- Hexadecimal

Base/radix = 2

The range of digits = 0 to (2-1) = 0 to 1

Base/radix = 8

The range of digits = 0 to (8-1) = 0 to 7

Base/radix = 10

The range of digits = 0 to (10-1) = 0 to 9

Base/radix = 16

The range of digits = 0 to (16-1) = 0 to 15

Quick Activity #0

Identify the radix for the following numbers:

1. 283
2. 54
3. 215
4. 2C
5. 1011

Positional Number Systems

- In a positional number system, each number represented by a string of digits in which each digit position i has an associated weight r^i, where r is the **radix**, or **base**, of the number system.

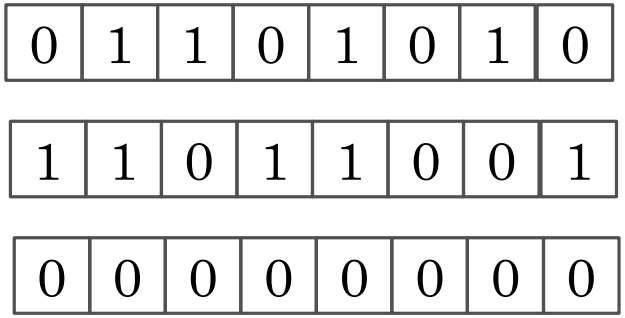
Position	4	3	2	1	0	−1
Value in Exponential Form	7^4	7^3	7^2	7^1	7^0	7^{-1}
Decimal Value	2401	343	49	7	1	1/7

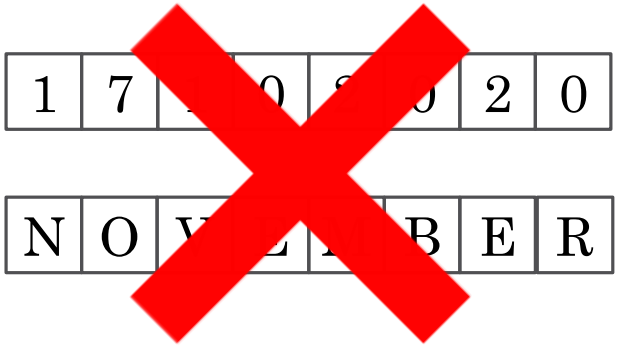
- The value of any digit ai is an integer in the range $0 \leq i < r$.
- The dot between a_0 and a_{-1} is called the radix point.
- As an example of another positional system, consider the system with base 7.

Introduction

- Bits in a computer system are grouped together into **cells**

An eight-bit memory cell in main memory





Decimal

- The decimal number system is also known as the base 10 system
- It is referred to as base 10 because it has 10 different symbols
- The based 10 system is also said to have radix of 10
- We use decimal systems with **decimal digits**
  - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

The Decimal System

Consider what the number 83 means?

Eight tens plus three $83 = (8 \times 10) + 3$

What about 4728?

Four thousands, seven hundreds, two tens, plus eight

$4728 = (4 \times 1000) + (7 \times 100) + (2 \times 10) + 8$

The Decimal System

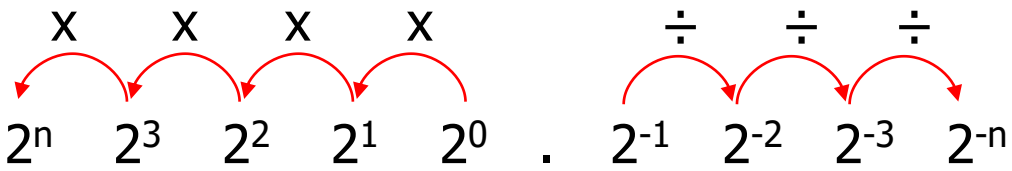
- A decimal system is a **base** or **radix** of **10**
  - The number multiply by 10 to the **power** of the **digit's position** (place value)

1 0

$83 = (8 \times 10^1) + (3 \times 10^0)$

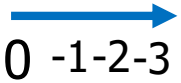
3 2 1 0

$4728 = (4 \times 10^3) + (7 \times 10^2) + (2 \times 10^1) + (8 \times 10^0)$



The Decimal System

- The same principles apply to **fraction** but with **negative powers** of 10s

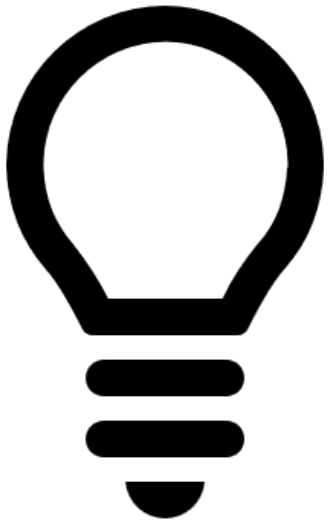


$0.256 = (2 \times 10^{-1}) + (5 \times 10^{-2}) + (6 \times 10^{-3})$

$442.256 = (4 \times 10^{2}) + (4 \times 10^{1}) + (2 \times 10^{0}) + (2 \times 10^{-1}) + (5 \times 10^{-2}) + (6 \times 10^{-3})$

Leftmost digit = Most Significant bit (MSB)

Rightmost digit = Least Significant bit (LSB)

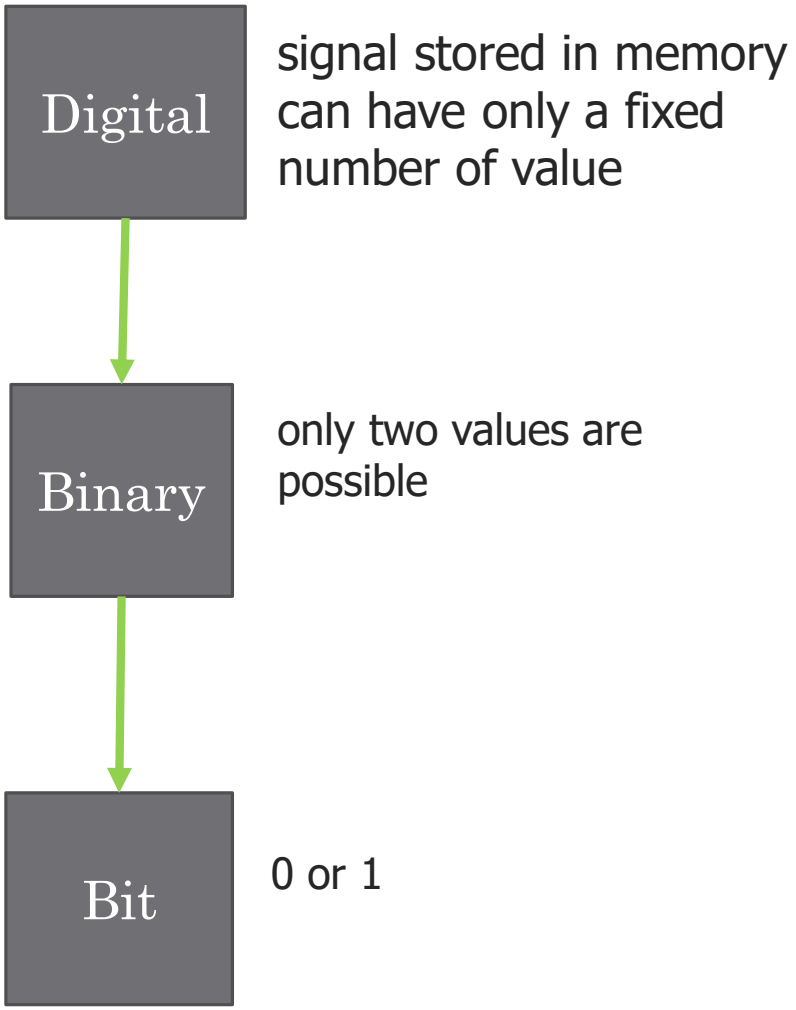


Quick research

How many tens are in the 509?

Introduction

- **Discrete** elements are represented by **signals**
  - Signals have two discrete values: **0** and **1**, binary values -> binary digit (*bit*)



The Binary System

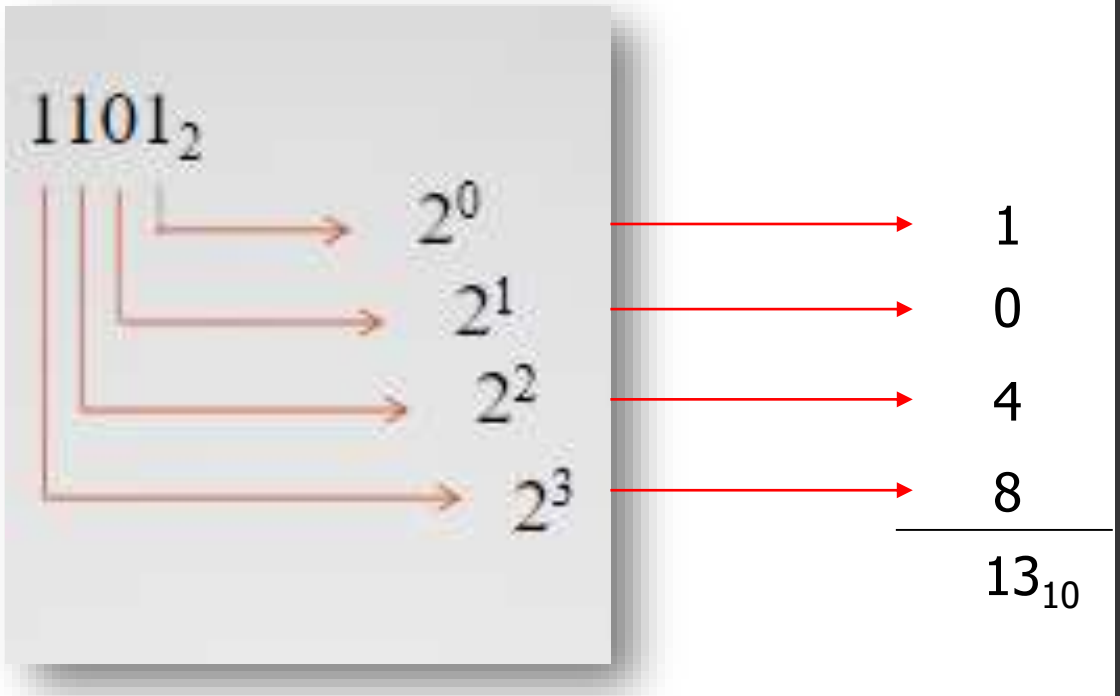
- To avoid confusion, we will sometimes put a subscript on a number to indicate its base
- For larger numbers, each digit in a binary number has a value depending on its **position** (as in decimal system)

$$0_2 = 0_{10}$$

$$1_2 = 1_{10}$$

The Binary System

- Binary number are strings of two (hence “bi”) symbols 0’s and 1’s, that represent numbers.
- 10 different digits are used to represent numbers with a base of 10
- There are only two digits, 1 and 0
  - Hence **base 2**



The terminology

BIT	• Each digit in the binary system
NIBBLE	• A group of 4 bits binary
BYTE	• A group of 8 bits binary
WORD	• Two bytes number

The Binary System

$2^0 = 1$

$10_2 = (1 \times 2^1) + (0 \times 2^0) = 2_{10}$

$11_2 = (1 \times 2^1) + (1 \times 2^0) = 3_{10}$

$100_2 = (1 \times 2^2) + (0 \times 2^1) + (0 \times 2^0) = 4_{10}$

$1001.102 = (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) + (0 \times 2^{-2}) + (2 \times 2^{-3}) = 9.75_{10}$

$1001.102 = (1 \times 2^3) + (1 \times 2^0) + (1 \times 2^{-1}) + (2 \times 2^{-3}) = 9.75_{10}$

In general, for the binary representation of $Y = 5c b_2 b_1 b_0 . b_{-1} b_{-2} b_{-3} c6$, the value of Y is

$$Y = \sum_i (b_i \times 2^i)$$

Spot the difference +

Decimal

$$X = \sum_i (d_i \times 10^i)$$

Binary

$$Y = \sum_i (b_i \times 2^i)$$

In a nutshell

- The binary system is closely tied to Boolean logic, a mathematical system used for representing and manipulating logical values (true and false)
- Binary is the natural language of digital electronics.
- Data storage in computers, including hard drives and memory, is organized into binary data structures.
- Overall, the binary system is fundamental to the operation of computers and digital technology, as it provides a simple and efficient way to represent and process information in a digital form.

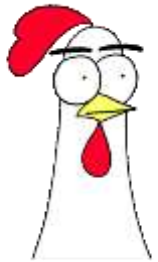
Converting From: Binary To Decimal

- **Multiply** each binary digit by the appropriate **power of 2** and add the results

$$b_{m-1}b_{m-2}\ldots b_2b_1b_0 \qquad b_i = 0 \text{ or } 1$$

$$(b_{m-1} \times 2^{m-1}) + (b_{m-2} \times 2^{m-2}) + \ldots + (b_1 \times 2^1) + b_0$$

Converting From: Binary To Decimal



$(a_3\ a_2\ a_1\ a_0.a_{-1}\ a_{-2})_2 \rightarrow (\quad)_{10}$

$(a_3 2^3 + a_2 2^2 + a_1 2^1 + a_0 2^0 + .a_{-1} 2^{-1} + a_{-2} 2^{-2})r \rightarrow (\quad)_{10}$

Example:

4 3 2 1 0 -1-2

$(10101.11)_2 \rightarrow (\quad \mathbf{21.75} \quad)_{10}$

1	a_4
0	a_3
1	a_2
0	a_1
1	a_0
1	a_{-1}
1	a_{-2}

$$(1x2^4) + (0x2^3) + (1x2^2) + (0x2^1) + (1x2^0) + (1x2^{-1}) + (1x2^{-2})$$

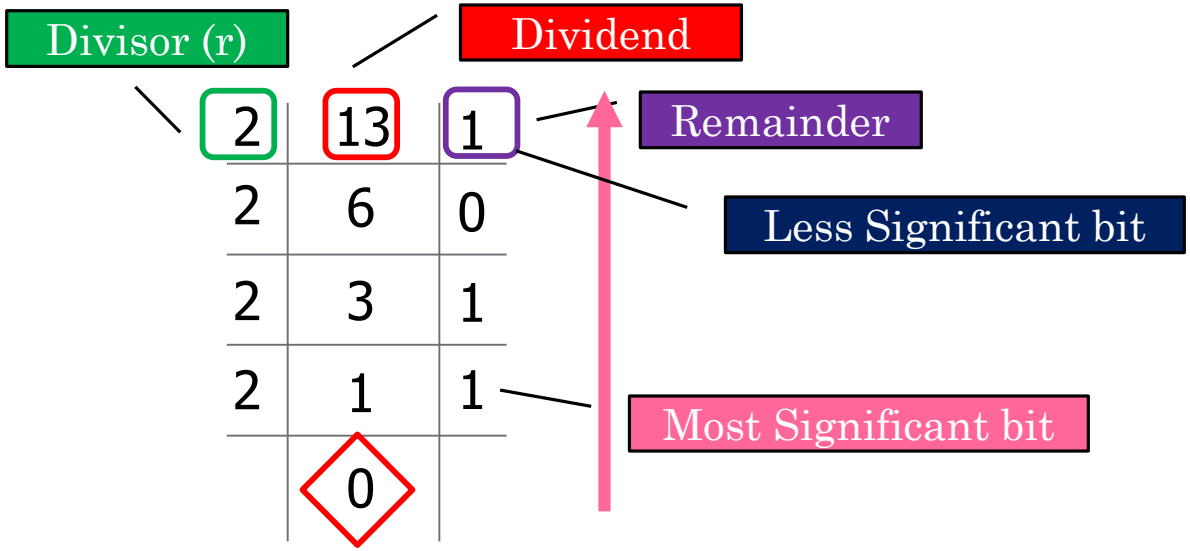
$\frac{1}{2^1}$ $\frac{1}{2^2}$

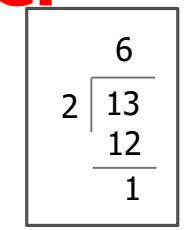
$$16 + 0 + 4 + 0 + 1 + 0.5 + 0.25$$

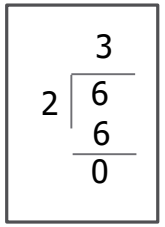
Converting From: Decimal To Binary (Integer)

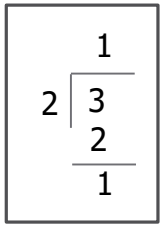
To convert decimal to any other base 'r', **divide integer** part by r and **multiply fractional** part by r

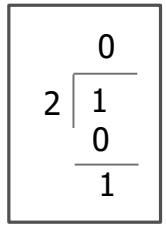
1. $(13)_{10} \rightarrow (1101)_2$











Converting From: Decimal To Binary (Integer)

- $(11)_{10}$ → $(\mathbf{1011})_2$

2	11	1
2	5	1
2	2	0
2	1	1
	0	

$$2 \overline{)\,11} \quad 5 \text{ R } 1$$

$$2 \overline{)\,5} \quad 2 \text{ R } 1$$

$$2 \overline{)\,2} \quad 1 \text{ R } 0$$

$$2 \overline{)\,1} \quad 0 \text{ R } 1$$

Converting From Decimal To Binary (Fractions)

1. $(25.625)_{10}$ → $(11001.101)_2$

Step 1 — **25** ÷ → Integer

Step 2 — **0.625** x → Fraction

2	25	1
2	12	0
2	6	0
2	3	1
2	1	1
	0	

11001

0.625 x 2 = 1.25

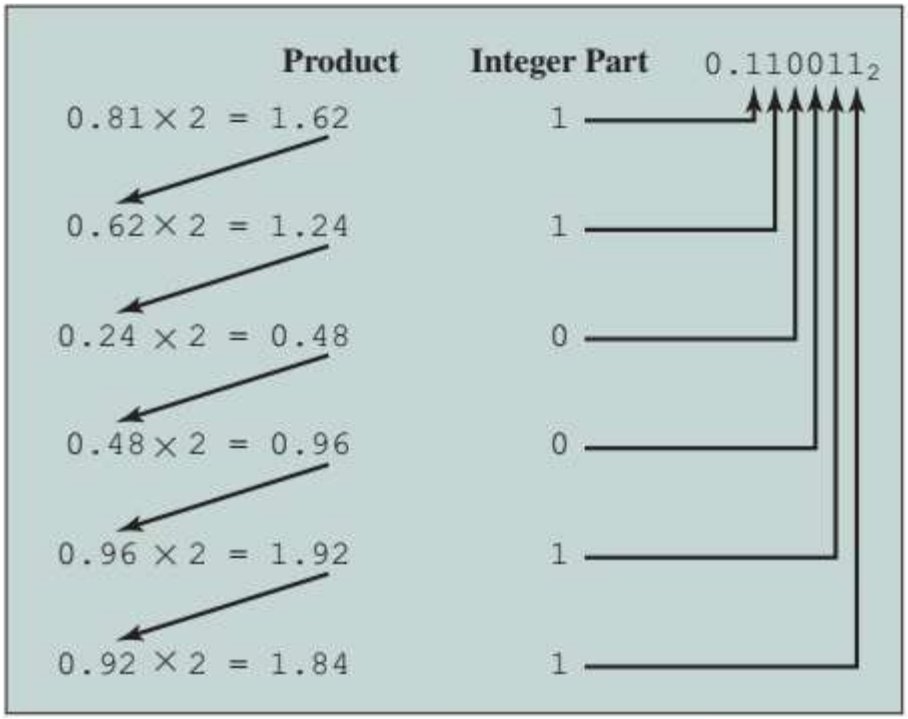
0.25 x 2 = 0.50

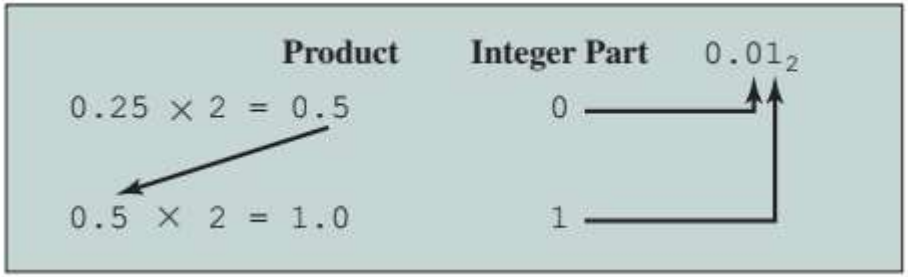
0.50 x 2 = 1.00

0.00 x 2 = 0.00

101

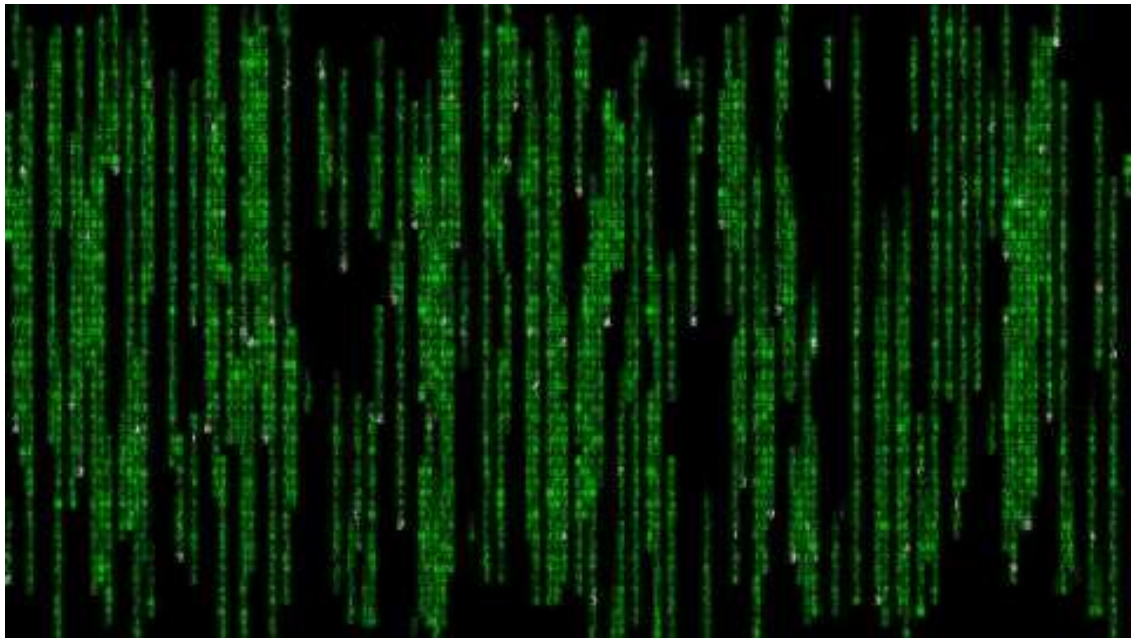
Converting From Decimal To Binary (Fractions)





The Hexadecimal Notation

- **Binary** notation is **cumbersome**
- Converting to decimal – **tedious**



r=16

The Hexadecimal Notation

- In hexadecimal number system, we have 16 different digits
- Each one of them can be represented by equivalent 4-bit binary number
- Better option is **hexadecimal** – **16 digits** 0…9, A, B, C, D, E, F

$$2C_{16} = (2_{16} \times 16^1) + (C_{16} \times 16^0)$$
$$= (2_{10} \times 16^1) + (12_{10} \times 16^0) = 44$$

The Hexadecimal Notation

- r = 16
- 0 will be the first digit
- Plus 15 other digits

	0	0000
	1	0001
	2	0010
	3	0011
	4	0100
	5	0101
	6	0110
	7	0111
	8	1000
	9	1001
(10)	A	1010
(11)	B	1011
(12)	C	1100
(13)	D	1101
(14)	E	1110
(15)	F	1111

The need for hexadecimal numbers

- In computing systems, the binary string equivalents of large decimal numbers can become quite long.
- When 16- or 32-bit numbers are involved, it becomes difficult to read and write them without producing errors.
- These problems can be overcome by arranging the binary numbers into groups of four bits, i.e., by using the hexadecimal numbering system.
- The format of hex numbers is more compact than binary numbers because they can represent large binary numbers with fewer digits.
- As a result, they are easier to understand than long binary strings of 1s and 0s.

Hexadecimal to Binary

	0	0000
	1	0001
	2	0010
	3	0011
	4	0100
	5	0101
	6	0110
	7	0111
	8	1000
	9	1001
(10)	A	1010
(11)	B	1011
(12)	C	1100
(13)	D	1101
(14)	E	1110
(15)	F	1111

1. $(259A)_{16}$ → $(0010\ 0101\ 1001\ 1010)_2$

2 = 0010
5 = 0101
9 = 1001
A = 1010

2. $(CAFE.3D)_{16}$ → $(1100\ 1010\ 1111\ 1110\ 0011\ 1101)_2$

C = 1100
A = 1010
F = 1111
E = 1110
3 = 0011
D = 1101

Binary to Hexadecimal

	0	0000
	1	0001
	2	0010
	3	0011
	4	0100
	5	0101
	6	0110
	7	0111
	8	1000
	9	1001
(10)	A	1010
(11)	B	1011
(12)	C	1100
(13)	D	1101
(14)	E	1110
(15)	F	1111

$(10001001.11)_2 \longrightarrow (\quad)_{16}$

1. Group the integer into 4 bits

10001001.11

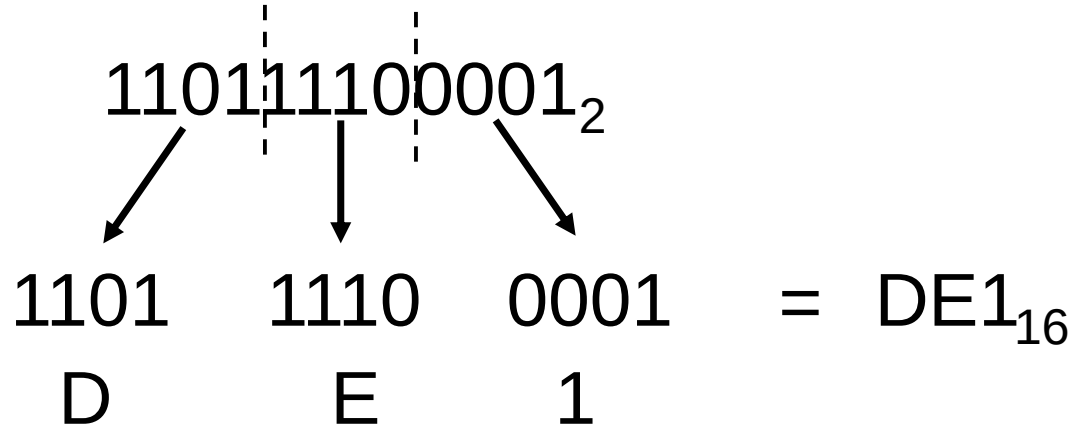
2. For the digit with insufficient bit, you will add 00 to complete the group of 4 bits

10001001.1100

$(89.C)_{16}$

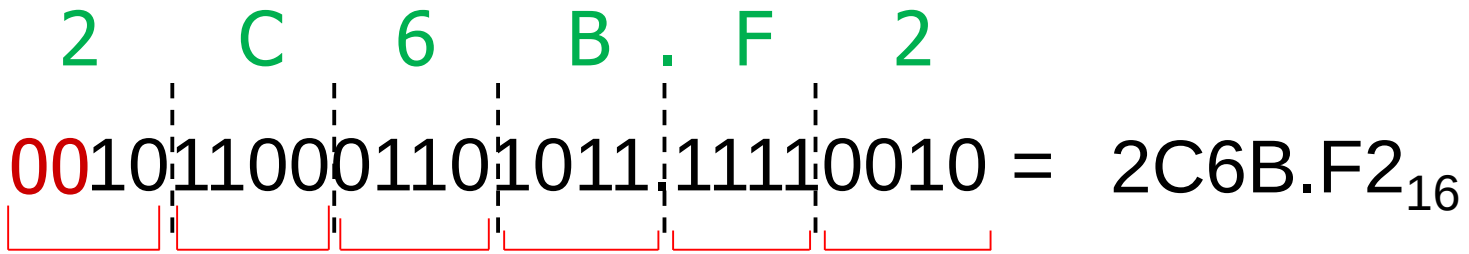
Binary to Hexadecimal

- The binary number is **divided** into **groups of four digits = nibble**



Binary to Hexadecimal

- The binary number is **divided** into **groups of four digits**



0000 = 0	0100 = 4	1000 = 8	1100 = C
0001 = 1	0101 = 5	1001 = 9	1101 = D
0010 = 2	0110 = 6	1010 = A	1110 = E
0011 = 3	0111 = 7	1011 = B	1111 = F

Decimal to Hexadecimal

$(479)_{10} \rightarrow (\,?\,)_{16}$

479 ÷ 16 = 29.9375 = 29 R 15

0.9375 x16 = 15

29 ÷ 16 = 1.8125 = 1 R 13

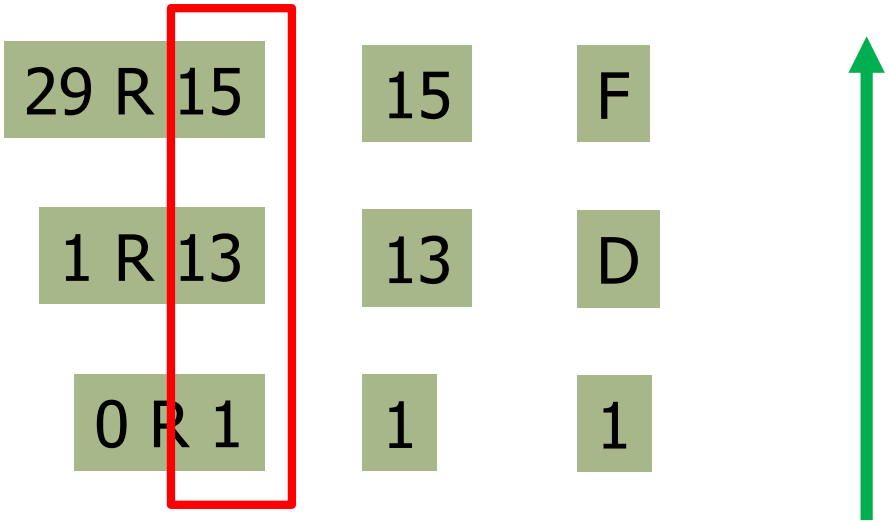
0.8125 x16 = 13

1 ÷ 16 = 0.0625 = 0 R 1

0.0625 x16 = 1

Decimal to Hexadecimal

$(479)_{10}$ → $(\ 1DF\ )_{16}$



Decimal (base 10)	Binary (base 2)	Hexadecimal (base 16)
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F
16	0001 0000	10
17	0001 0001	11
18	0001 0010	12
31	0001 1111	1F
100	0110 0100	64
255	1111 1111	FF
256	0001 0000 0000	100

In a nutshell

- Hexadecimal notation is not only used for representing integers but also used as a concise notation for representing any sequence of binary digits, whether they represent text, numbers, or some other type of data
- The reasons for using hexadecimal notation:
  - ❖It is more compact than binary notation.
  - ❖In most computers, binary data occupy some multiple of 4 bits, and hence some multiple of a single hexadecimal digit.
  - ❖It is extremely easy to convert between binary and hexadecimal notation.

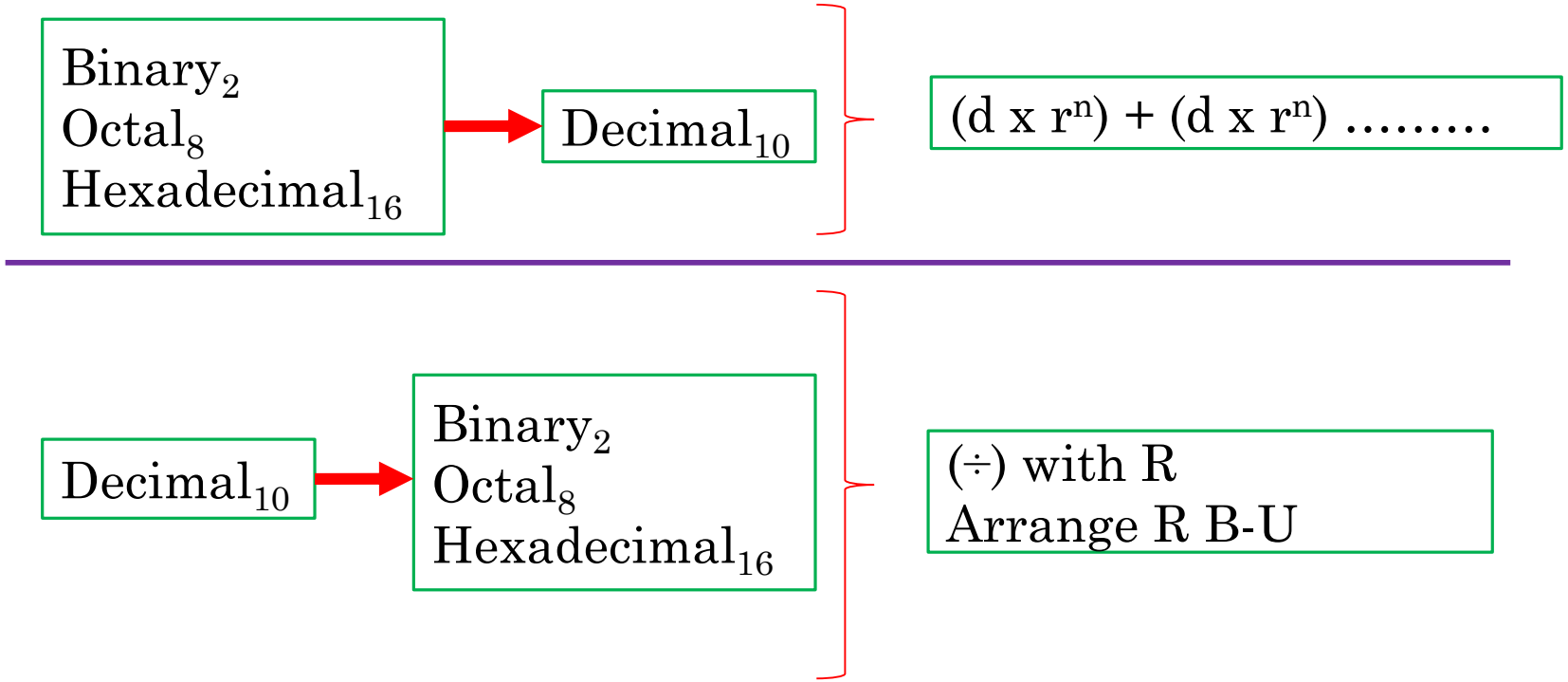
567_8

- Convert the above octal number to decimal.

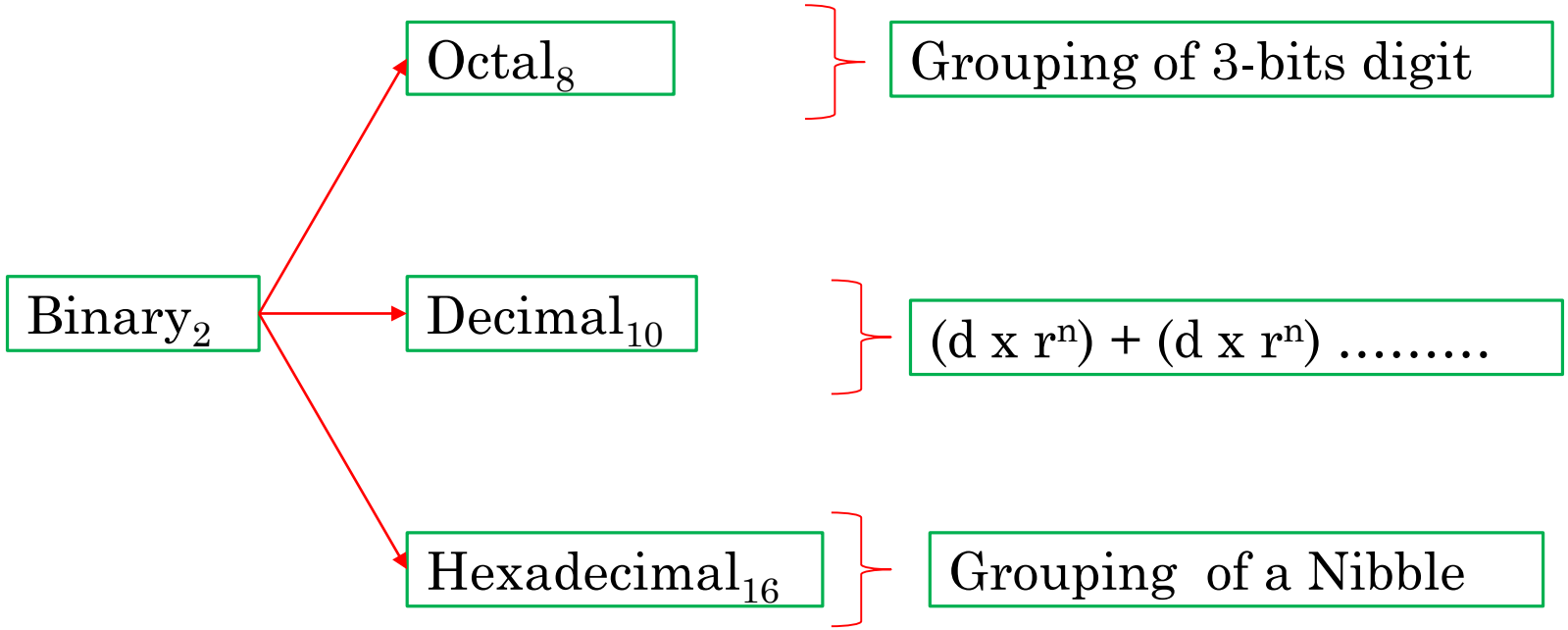
567_8

- Convert the above octal number to decimal.
- What numbers did you add to get the answer?

Conversion summary



Conversion summary



$A98_{16}$

- Convert the above hexadecimal number to octal.

Character Codes : BCD

- Decimal digits
  - **Binary Coded Decimal** (BCD) code

Binary Power	2^3	2^2	2^1	2^0
Binary Weight:	8	4	2	1

Decimal digit	BCD code
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Character Codes : ASCII

ASCII stands for American Standard Code for Information Interchange

- 7-bit **ASCII** codes (American Standards Committee on Information Interchange)
- ASCII codes represent text in computers, communications equipment, and other devices that use text
- This code is basically used for identifying characters and numerals in a keyboard.
- ASCII is a character encoding method that uses a 7-bit code to represent 128 characters, which include letters, numbers, and special characters.

ASCII Code

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Character Codes : EBCDIC

- An 8-bit character encoding used mainly on IBM mainframe and IBM midrange computer operating systems
- EBCDIC has 256 characters and is used to represent a wide variety of characters, such as letters, numbers, special characters, and control characters.

Character Codes : EBCDIC

Character	EBDIC Bit Configuration	Character	EBDIC Bit Configuration
A	1100 0001	S	1110 0010
B	1100 0010	T	1110 0011
C	1100 0011	U	1110 0100
D	1100 0100	V	1110 0101
E	1100 0101	W	1110 0110
F	1100 0110	X	1110 0111
G	1100 0111	Y	1110 1000
H	1100 1000	Z	1110 1001
I	1100 1001	0	1111 0000
J	1101 0001	1	1111 0001
K	1101 0010	2	1111 0010
L	1101 0011	3	1111 0011
M	1101 0100	4	1111 0100
N	1101 0101	5	1111 0101
O	1101 0110	6	1111 0110
P	1101 0111	7	1111 0111
Q	1101 1000	8	1111 1000
R	1101 1001	9	1111 1001

Reflection…

Watch video on Format Wars:
ASCII vs EBCDIC.
Who won/lost and why?

Link on eLearn / Chat

Image source: https://www.flickr.com/photos/29143375@N05/4582236533


Format Wars
ASCII
EBCDIC